

COMPENDIO TÁCTICO: PROGRAMACIÓN DINÁMICA

Ecuaciones de Bellman, Tabulación y Catálogo de Patrones Óptimos (Edición Completa)

1. Estructura y Conceptos Fundamentales de la PD

La Programación Dinámica no es magia, es fuerza bruta inteligente. Para justificar formalmente en un examen que un problema exige y permite PD, debes citar y explicar estas dos propiedades innegociables:

Propiedades Vitales (El Escudo Teórico)

- **Subestructura Óptima (Principio de Optimalidad de Bellman):** La solución óptima al problema global se puede construir componiendo las soluciones óptimas de sus subproblemas más pequeños. Esto implica que cualquier estado intermedio en el camino hacia la solución global debe ser, por definición, una solución óptima para ese punto temporal.
- **Superposición de Subproblemas:** El espacio de subproblemas es "pequeño" repetitivo. Al aplicar la recursividad pura (Divide y Vencerás), el árbol de llamadas calcularía los mismos nodos una y otra vez, disparando el tiempo a $\mathcal{O}(2^N)$ o $\mathcal{O}(N!)$. La PD soluciona esto dotando al algoritmo de "memoria" para que ningún estado se calcule más de una vez.

Los dos Enfoques de Implementación (Memorización vs Tabulación)

- **Top-Down (Memoización):** Se programa de forma recursiva natural, exactamente igual que un Divide y Vencerás, pero se inyecta una matriz global al inicio. Al entrar a la función, se comprueba: si $M[i][j]$ ya tiene un valor, se devuelve instantáneamente (poda de la rama). Es más intuitivo de programar, pero sufre de sobrecarga por llamadas a la pila de recursión (riesgo de *StackOverflow*).
- **Bottom-Up (Tabulación):** Se eliminan todas las llamadas recursivas. Se construye una tabla iterativa empezando por los casos base más pequeños y triviales, y se rellena mediante bucles **for** anidados hasta alcanzar la celda que representa el problema general. Es más eficiente y es el enfoque exigido por defecto en los exámenes universitarios.

2. Metodología de Resolución: Los 5 Pasos Innegociables

Resolver un problema de PD es rellenar una plantilla estricta. Todo ejercicio que entregues debe detallar los siguientes 5 pasos. Fallar en el Paso 1 condena automáticamente a los otros cuatro.

■ PASO 1: Definición del Subproblema y la Tabla (El Significado de la Matriz)

- *La Lógica:* Tienes que explicar en lenguaje natural qué representan los ejes (filas y columnas) de tu tabla y, sobre todo, qué almacena cada celda. Si te equivocas aquí, la ecuación de recurrencia no saldrá.
- *Plantilla:* "Sea $T[i][j]$ el [beneficio máximo / coste mínimo / cantidad de formas posibles] de resolver el problema si consideramos únicamente un subconjunto desde el elemento 1 hasta el índice i , disponiendo de una capacidad/restricción máxima de valor j ."

■ PASO 2: Ecuación de Recurrencia (La Ecuación de Bellman)

- *La Lógica:* Es el motor matemático. Es la fórmula que calcula el valor de una celda combinando el valor de celdas previamente calculadas. Siempre implica evaluar el abanico de decisiones que puedes tomar en ese instante (ej. coger un objeto o ignorarlo) y aplicar un operador (máx, mín, o un sumatorio $\sum$).
- *Estructura:* $T[i][j] = \text{máx} / \text{mín} \begin{cases} \text{Decisión A (Ignorar)} \rightarrow T[i-1][j] \\ \text{Decisión B (Usar)} \rightarrow T[i-1][j - \text{peso}] + \text{beneficio} \end{cases}$

■ PASO 3: Casos Base (Condiciones de Contorno de la Tabla)

- *La Lógica:* La ecuación de recurrencia mira hacia celdas anteriores ($i-1$, $j-x$). Para evitar que los bucles **for** intenten acceder a índices negativos (y causen un error de segmentación), debemos precargar las primeras filas y/o columnas con valores lógicos triviales.
- *Ejemplos Vitales:*
 - $T[0][j] = 0$: "Si no tengo objetos que procesar, el beneficio obtenido es 0".
 - $T[i][0] = \infty$: "Si intento minimizar un coste pero mi capacidad es 0, marco la celda con infinito para indicar que ese estado es imposible/inválido".

■ PASO 4: Orden de Evaluación (La Dirección del Barrido)

- *La Lógica:* Equivalente a la ordenación topológica de un grafo de dependencias. Tienes que definir cómo los bucles **for** barren la tabla para asegurar que, cuando vayas a calcular $T[i][j]$, las celdas a las que llamas en tu Paso 2 ya hayan sido calculadas iteraciones atrás.
- *Ejemplos:* "La matriz se evalúa fila a fila, de arriba hacia abajo ($i = 1 \dots N$). Dentro de cada fila, se evalúa de izquierda a derecha ($j = 1 \dots W$). Las recurrencias por intervalos se evalúan por diagonales de longitud creciente."

■ PASO 5: Reconstrucción de la Solución (El Traceback)

- *La Lógica:* La celda final $T[N][W]$ solo contiene un número (ej. el beneficio máximo fue 150€). ¡Pero el problema exige saber **qué objetos** has cogido para ganar esos 150€!
- *Mecánica:* Debemos empezar en la celda final y hacer un recorrido inverso aplicando la lógica: "¿De dónde sacó esta celda su valor?". Si $T[i][j]$ tiene exactamente el mismo valor que $T[i-1][j]$ (la celda de arriba), significa que el objeto i NO se usó. Si el valor es distinto, significa que sí se usó; registramos el objeto i y pegamos un salto diagonal a la celda $T[i-1][j - \text{peso}_i]$ para seguir retrocediendo.

3. Plantillas Mentales por Patrón (La Guía Rápida del Examen)

Al enfrentarte a un problema nuevo, el primer paso es encajarlo en uno de estos 4 "Patrones Maestros". Cada patrón define de antemano cuántas dimensiones tendrá tu tabla y qué forma tendrá la recurrencia.

■ A. Patrón Unidimensional Clásico (Avanzar por un Array)

- *Cuándo usar:* El problema te da una secuencia lineal y debes tomar decisiones saltando por ella sin límite de presupuesto. Ejemplos: Subsecuencia Creciente más Larga (LIS), Problema del Salto de la Rana, Robo de casas contiguas.
- *Definición y Tabla:* 1 Dimensión. $dp[i]$ = La mejor solución si el recorrido termina obligatoriamente en el elemento i .
- *Transición típica:* Mirar hacia atrás en la misma dimensión. $dp[i] = \max(dp[i-1], dp[i-2] + \text{valor}[i])$.

■ B. Patrón de Capacidad / Tipo Mochila (0-1 y Variantes Multidimensionales)

- *Cuándo usar:* El problema dice "Tienes N candidatos y W recursos máximos (espacio, dinero, tiempo). Elige un subconjunto".
- *Definición y Tabla:* 2 Dimensiones. $dp[i][w]$ = El estado óptimo usando un subconjunto de los objetos del 1 al i , consumiendo como máximo w recursos.
- *Transición típica:* Decisión dicotómica (lo ignoro vs. lo uso, si cabe).

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{peso}[i]] + \text{valor}[i])$$

■ C. Patrón de Prefijos en Dos Secuencias (Strings y Textos)

- *Cuándo usar:* El enunciado te da DOS cadenas o vectores y pide compararlos, alinearlos o editarlos. Ejemplos: LCS, Distancia de Edición (Levenshtein), Bioinformática (ADN).
- *Definición y Tabla:* 2 Dimensiones. $dp[i][j]$ = Solución óptima para comparar el **prefijo** de tamaño i de la primera cadena, con el **prefijo** de tamaño j de la segunda.
- *Transición típica:* Evaluar si los caracteres actuales coinciden o no.

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{si } A[i] == B[j] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{si } A[i] \neq B[j] \end{cases}$$

■ D. Patrón de Intervalos Desvinculados (Cortar y Pivotes)

- *Cuándo usar:* El problema exige procesar una secuencia, pero las decisiones óptimas implican partir la secuencia por el medio, resolviendo mitades independientes. Ejemplos: Multiplicación de Matrices, Árboles ABBO, Triangulación de Polígonos.
- *Definición y Tabla:* 2 Dimensiones de índices. $dp[i][j]$ = Solución óptima para procesar el **sub-intervalo estricto** que empieza en el índice i y termina en el índice j .
- *Transición típica:* Requiere un bucle interior k que actúe de cuchillo pivotante, troceando el intervalo $[i, j]$ en $[i, k]$ y $[k+1, j]$.

$$dp[i][j] = \min_{i \leq k < j} \{ dp[i][k] + dp[k+1][j] + \text{Coste_de_unir}(i, k, j) \}$$

4. Justificaciones Formales de Corrección y Argumentación

En los exámenes, es habitual que pidan demostrar la corrección de tu solución. Usa estas vías lógicas redactadas de forma académica.

Vía 1: Demostración por Contradicción (Para probar la Subestructura Óptima)

1. Supongamos que hemos construido una solución global S al problema que es innegablemente la solución óptima.
2. Si analizamos S y extraemos el último paso lógico tomado (por ejemplo, la decisión de incluir o excluir el último objeto N), el gran problema original queda escindido en un subproblema de dimensiones menores (ej. objetos $N - 1$). Denotaremos la solución contenida dentro de S para ese subproblema como S_{sub} .
3. Supongamos por reducción al absurdo que S_{sub} **no** representa la resolución óptima para ese tamaño, y que existiera hipotéticamente otra configuración S'_{sub} que devolviera un beneficio estrictamente mayor.
4. Si sustituyésemos el fragmento S_{sub} por la configuración superior S'_{sub} y le volviésemos a concatenar la última decisión extraída en el paso 2, habríamos forjado una solución global S' cuyo beneficio total sería superior al de S .
5. Esto entra en colisión frontal con nuestra premisa del paso 1 de que S era la solución óptima absoluta.
6. Ergo, la solución óptima de un problema incluye ineludiblemente las soluciones óptimas de sus subproblemas. El diseño de nuestra Ecuación de Bellman, que combina subproblemas ya optimizados, es matemáticamente correcto.

Vía 2: Inducción Matemática sobre los Estados (Para probar la Tabulación)

1. **Caso Base:** Las celdas de inicialización (ej. $dp[0][w] = 0$) contienen resultados trivialmente correctos por la propia semántica del problema (ej. el beneficio es nulo si no se procesa ningún elemento).
2. **Hipótesis Inductiva:** Asumimos como axioma temporal que, al hallarnos calculando un estado actual (i, j) , todas las celdas (i', j') calculadas en instantes anteriores (aquellas donde $i' < i$ o $j' < j$) ya albergan en memoria la solución garantizada y óptima a sus respectivos subproblemas acotados.
3. **Paso Inductivo:** La ecuación de recurrencia ejecutada sobre $dp[i][j]$ opera forzando una evaluación exhaustiva (vía operador máx o mín) de la totalidad de ramificaciones de decisión posibles en ese preciso instante. Dado que (por la hipótesis previa) los operandos extraídos de la tabla para estos cálculos son 100 % óptimos, la extensión paramétrica de los mismos heredarán inexorablemente esa optimalidad.
4. **Conclusión:** Al proseguir con el orden de evaluación dictado, el último estado $dp[N][W]$ encapsulará una solución global inobjetable.

Vía 3: ¿Por qué falla un algoritmo Voraz (Greedy) en estos casos?

Si el profesor pregunta por qué este problema no se puede resolver con Voraces, ataca directamente a las consecuencias a largo plazo:

- .El paradigma Voraz (Greedy) requiere incondicionalmente que una elección considerada óptima a nivel local conduzca de forma irreversible y directa al óptimo global, sin requerir reevaluación ni retrospectiva. Sin embargo, en el problema que nos atañe, las decisiones tomadas en pasos tempranos sufren de **severas consecuencias de largo plazo** que un enfoque miope no puede prever.”
- .Al decantarnos por el candidato aparentemente más rentable en el instante i (por ejemplo, escogiendo la denominación de moneda más alta o el objeto con el mejor ratio beneficio/peso), corremos el riesgo extremo de agotar prematuramente un recurso crítico (como la capacidad residual de la mochila o el montante del cambio). Este acaparamiento de recursos genera un bloqueo sistémico, imposibilitando que el algoritmo aproveche combinaciones futuras de elementos más pequeños que, en su conjunto, brindarían un beneficio global inmensamente superior.”
- ”La Programación Dinámica sorteja este abismo metodológico explorando de forma implícita y garantizada **todas las ramificaciones de decisión existentes** (incluir el candidato vs. excluirlo), pero eviscerando la explosión combinatoria (tiempo exponencial) gracias al almacenamiento tabular de los subproblemas precalculados.”

5. Análisis de Complejidades (Tiempo y Espacio)

Para clavar la puntuación en la justificación de la complejidad de un algoritmo de PD, recurre siempre a esta fórmula fundacional:

Tiempo Total = (Número total de Estados) $\times$ (Coste computacional de procesar un Estado)

- **Ejemplo Mochila 0-1:** Dimensionamos la tabla en N filas y W columnas. Total de estados = $N \times W$. En cada iteración, computar el estado demanda exclusivamente invocar una función $\max(A, B)$ y realizar lecturas directas en matriz, lo cual tiene un coste de $\mathcal{O}(1)$. Por consiguiente, $\text{Tiempo} = \mathcal{O}(N \cdot W)$. (A esta complejidad se le denomina técnica de tiempo *pseudo-polinómico*, puesto que W es un valor numérico y no el tamaño en bits de la entrada).
- **Ejemplo Cadena de Matrices:** Contemplamos $\approx N^2/2$ estados útiles (el triángulo superior de una matriz $N \times N$). Para finiquitar la ecuación de cada estado individual, empleamos un bucle interno k que itera en el peor de los casos $\mathcal{O}(N)$ veces buscando el pivote de corte. Por tanto, $\text{Multiplicación} = N^2 \times N = \mathcal{O}(N^3)$.

Reducción Drástica del Espacio (El truco del Rolling Array.º Vector Rotatorio):

Explicar esto demuestra un conocimiento absoluto de la asignatura. En problemas como la Mochila o el LCS, si te fijas meticulosamente en la Ecuación de Bellman, notarás que $dp[i][w]$ **exclusivamente realiza operaciones de lectura sobre los datos alojados en la fila cronológica inmediatamente anterior** ($dp[i-1][\dots]$). Las filas $i-2$, $i-3$, etc., jamás vuelven a ser consultadas una vez superadas. Por este motivo ingenieril, resulta ineficiente e innecesario almacenar la matriz íntegra de dimensiones $N \times W$ en la memoria RAM. Es plausible y factible programar el algoritmo reservando memoria para un único array unidimensional de tamaño W . Este array se actualizará sobre sí mismo en cada paso del bucle exterior (recorriéndolo siempre de derecha a izquierda, de W hacia 0, para evitar el autoboot de sobrescribir datos que aún necesitamos procesar en la misma iteración). Esta optimización de élite comprime la complejidad espacial de $\mathcal{O}(N \cdot W)$ a $\mathcal{O}(W)$. (Nota: Si aplicamos esta técnica destructiva, sacrificamos la capacidad de ejecutar el proceso de Reconstrucción (Traceback) completo, obteniendo únicamente el valor numérico óptimo final).

6. Desarrollo Analítico de los 6 Problemas Canónicos (Los 5 Pasos en Acción)

Esta es la artillería pesada. Aquí están desglosados los grandes clásicos de la PD con todo el nivel de detalle exigido en un examen, explicando el "por qué" se toman ciertas decisiones.

A. El Problema del Cambio de Monedas (Versión General con Cantidades Ilimitadas)

Contexto: Voraz falla aquí si el sistema de monedas es "1, 5, 12€ te piden 15. Voraz escogería 12+1+1+1 (4 monedas). Dinámica escogerá 5+5+5 (3 monedas).

- **Paso 1 (Subproblema):** Sea $T[c]$ la **cantidad mínima** de monedas indispensables para devolver un montante exacto de c céntimos, disponiendo de un conjunto de denominaciones $v = \{v_1, \dots, v_n\}$ inagotables.
- **Paso 2 (Recurrencia):** Para forjar la cantidad c , probamos someter a prueba todas y cada una de las monedas v_i del catálogo. Si extraemos la moneda v_i , el problema colapsa mágicamente a resolver cuánto cuesta devolver la cantidad residual $(c - v_i)$. El coste total será el del subproblema +1 (la moneda que acabamos de poner). Nos quedamos con la opción que escupa el mínimo:

$$T[c] = \min_{\forall i \text{ tal que } v_i \leq c} \{1 + T[c - v_i]\}$$

- **Paso 3 (Casos Base):** El pilar es $T[0] = 0$ (Se erogan 0 monedas para devolver 0 euros). Todo el resto del array de subproblemas $T[1 \dots C]$ debe preinicializarse con el valor centinela ∞ (infinito), para neutralizar que un estado irresoluble sea escogido erróneamente por el operador mín.
- **Paso 4 (Orden de Evaluación):** Un sencillo bucle lineal que avanza progresivamente c desde 1 hasta la capacidad objetivo C .
- **Paso 5 (Complejidad y Reconstrucción):** Complejidad Temporal $\mathcal{O}(n \cdot C)$ y Espacial $\mathcal{O}(C)$. Para habilitar el *traceback*, fundamos un vector adjunto $M[c]$ donde, cada vez que el mín se actualice en la celda $T[c]$ merced a la moneda v_i , grabaremos $M[c] = v_i$. Al finalizar, partiendo de C , la primera moneda de la solución es $M[C]$. Saltamos a la celda $C - M[C]$ y repetimos hasta tocar fondo en 0.

B. El Problema de la Mochila 0-1 (Knapsack)

Contexto: Maximizar el valor de los objetos robados sin reventar la capacidad de peso de la mochila. El paradigma es 0-1 porque un objeto no se puede astillar (te lo llevas íntegro o lo rechazas).

- **Paso 1 (Subproblema):** Sea $T[i][w]$ el valor de mercado **máximo** recabado examinando exclusivamente los objetos comprendidos del índice 1 al i , bajo una limitación de peso estricta w .
- **Paso 2 (Recurrencia):** Al contemplar el objeto i -ésimo, acaecen dos realidades mutables. Primero: si el objeto por sí solo es más pesado que la capacidad analizada ($peso_i > w$), es físicamente inadmisibile. Su rechazo es obligatorio,

por lo que heredamos el beneficio intacto de los objetos anteriores: $T[i][w] = T[i-1][w]$. Segundo: si gozamos del espacio suficiente para albergarlo, se abren dos universos de decisión, y el operador $\max$ nos guiará al óptimo:

$$T[i][w] = \max \begin{cases} T[i-1][w] & \text{(Decisión cobarde: Descartarlo)} \\ T[i-1][w - \text{peso}_i] + \text{valor}_i & \text{(Decisión audaz: Cogerlo. Consumimos peso y sumamos valor)} \end{cases}$$

- **Paso 3 (Casos Base):** La Fila 0 dicta que $T[0][w] = 0$ para cualquier lindeza de w (un botín de 0 objetos engendra 0 valor). La Columna 0 sentencia que $T[i][0] = 0$ (un espacio volumétrico nulo restringe el valor a 0).
- **Paso 4 (Orden de Evaluación):** Disposición clásica bidimensional. Un Bucle foráneo rige los objetos (i desde 1 a N), y un bucle intestinal modula los pesos (w desde 1 a W). La tabla se riega fila a fila, y en cada iteración horizontal, de babor a estribor.
- **Paso 5 (Complejidad y Reconstrucción):** Complejidad asintótica $\mathcal{O}(N \cdot W)$. Para desenmarañar la cesta de objetos, el rastreo (traceback) arranca en el pináculo $T[N][W]$. La prueba del algodón es la siguiente: comprobamos si $T[i][w] \neq T[i-1][w]$. Una asimetría certifica fehacientemente que el objeto i fue engullido por la mochila (puesto que inyectó un valor diferencial). Si es así, lo imprimimos por pantalla y retrocedemos a la celda $T[i-1][w - \text{peso}_i]$. Si por el contrario hay simetría de valores, lo desdeñamos y simplemente ascendemos una fila a $T[i-1][w]$.

C. Multiplicación Óptima de Secuencia de Matrices

Contexto: Agrupar matrices con paréntesis para minimizar el brutal número de operaciones aritméticas derivadas de sus variopintas dimensiones.

- **Paso 1 (Subproblema):** Sea $M[i][j]$ la cuantía **mínima** absoluta de multiplicaciones escalares exigidas para consumir el producto del bloque de matrices continuas $A_i \dots A_j$. Las cotas dimensionales de cada matriz individual k se rigen bajo el formato $d_{k-1} \times d_k$.
- **Paso 2 (Recurrencia):** La estratagema reside en ubicar un punto de escisión "pivote" (denominado k) que diseccione la secuencia matricial en dos mitades autónomas ($[i \dots k]$ y $[k+1 \dots j]$). El coste final emergerá de la sumatoria de organizar la mitad izquierda, sumar la organización de la derecha, y finalmente facturar el sobre coste ineludible de ensamblar ambos super-bloques:

$$M[i][j] = \min_{\forall k \text{ tal que } i \leq k < j} \{M[i][k] + M[k+1][j] + (d_{i-1} \cdot d_k \cdot d_j)\}$$

- **Paso 3 (Casos Base):** El axioma recae en $M[i][i] = 0$ (Multiplicar de forma aislada una única matriz contra la nada devenga en 0 operaciones y 0 esfuerzo).
- **Paso 4 (Orden de Evaluación): ¡Alerta Roja en Examen!** Si tratas de iterar esto en un esquema ordinario fila a fila, colapsará miserablemente, pues intentarás convocar sub-segmentos que aún yacen vacíos. La evaluación demanda **crecimiento longitudinal iterativo**. Se instauran bucles que construyen primero segmentos de longitud $L = 2$, luego $L = 3$, progresando hasta $L = N$. Visualmente, la matriz se repuebla trazando diagonales sucesivas que se van apartando paulatinamente de la diagonal principal nula.
- **Paso 5 (Complejidad y Reconstrucción):** Debido a la trinidad de bucles (L , i , y el cuchillo pivote k), el peso algorítmico se fija en $\mathcal{O}(N^3)$. Para el despliegue visual de los paréntesis, es imperativo salvaguardar el índice glorioso k que propició el mínimo dentro de una matriz auxiliar $K[i][j]$. El traceback es una delicia recursiva: se acude a la celda general, se lee el pivote, y la función se invoca a sí misma escindiéndose hacia el flanco izquierdo y derecho.

D. Subsecuencia Común Más Larga (LCS)

Contexto: La piedra roseta de la bioinformática comparativa y los softwares de control de versiones. Buscar retazos intactos entre dos flujos de caracteres.

- **Paso 1 (Subproblema):** Sea $L[i][j]$ la longitud paramétrica de la subsecuencia inquebrantable más extensa afín al prefijo recortado de tamaño i de la cadena textual X , y simultáneamente al prefijo de tamaño j de la cadena textual Y .
- **Paso 2 (Recurrencia):** En cada cruce de coordenadas observamos las letras terminales. La dialéctica dicta:
 - Escenario de Simetría ($X[i] == Y[j]$): Hemos cazado una letra de la subsecuencia maestra. Reclamamos este +1 al tesoro y delegamos el rastreo residual a la estela en diagonal de ambos strings mermados. $\rightarrow L[i][j] = L[i-1][j-1] + 1$
 - Escenario de Asimetría ($X[i] \neq Y[j]$): Un colapso parcial. Debemos sacrificar heurísticamente un carácter de algún frente para continuar la pugna. Invocamos al operador máximo para elegir el mal menor: amputamos del texto X o amputamos del texto Y . $\rightarrow L[i][j] = \max(L[i-1][j], L[i][j-1])$
- **Paso 3 (Casos Base):** Filas y columnas de inicialización cero: $L[0][j] = 0$ y $L[i][0] = 0$ (El cruce de un prefijo inexistente, de cadena vacía, con cualquier texto concebible, arroja inherentemente una comunión nula).
- **Paso 4 (Orden de Evaluación):** Esquema clásico de barrido de matriz bidimensional (Iterador exógeno para letras de X , iterador endógeno para letras de Y).

- **Paso 5 (Complejidad y Reconstrucción):** Complejidad asintótica temporal fijada en $\mathcal{O}(|X| \cdot |Y|)$. Para desenterrar las letras literales de la subsecuencia, el traceback germina en el extremo inferior derecho. Si las letras vectoriales coinciden $X[i] == Y[j]$, ese carácter es un soldado válido de la LCS; se engrosa el string final y se retrocede flagrantemente en diagonal pura hacia $[-1, -1]$. En caso adverso, la lógica nos impele a navegar hacia la celda lindante (norte u oeste) que exhiba una réplica exacta del valor métrico actual, perpetuando el rastreo.

E. Caminos Mínimos Globales (Algoritmo de Floyd)

Contexto: Buscar la red de autopistas más rápida entre todas las metrópolis de un mapa, asumiendo peajes incluso con saldo a favor (pesos negativos).

- **Paso 1 (Subproblema):** Sea $D^k[i][j]$ la longevidad kilométrica de la ruta **más sucinta** trazable desde el bastión i hasta el bastión j , coartada a transitar exclusiva y rígidamente utilizando como enclaves de intermediación y escala un subgrupo privilegiado de nodos $\{1, 2, \dots, k\}$.
- **Paso 2 (Recurrencia):** El epicentro cognitivo de Floyd. Al dar luz verde para que el nodo soberano k actúe como un nexo lícito, se interroga a la matriz: ¿La senda preestablecida antigua es óptima, o resulta rentablemente superior desviar el tráfico colateralmente efectuando una escala en el recién ungido nodo k ?

$$D^k[i][j] = \min(D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j])$$

- **Paso 3 (Casos Base):** La matriz seminal $D^0[i][j]$ es simplemente una transcripción fotográfica del peso crudo y directo de la arista topológica entre i y j (∞ infinito ante la ausencia física de vial, 0 frente al estatus de bucle $i = j$).
- **Paso 4 (Orden de Evaluación):** Una santa trinidad de bucles iterativos. El bucle majestuoso y englobador regula el pivote escalar de expansión k (desde 1 hasta N). Acunados en sus adentros, los bucles de cuadrícula i y j .
- **Paso 5 (Complejidad y Reconstrucción):** Coste inalterable de $\mathcal{O}(N^3)$. En pos de guiar al viajero visualmente nodo a nodo, se anida en sincronía una matriz satelital $R[i][j] = k$, destinada a cincelar en granito la identidad del último nodo empleado transaccionalmente como puente benefactor.

F. Árboles Binarios de Búsqueda Óptimos (ABBO)

Contexto: Armar un esqueleto de búsquedas donde los datos más solicitados estadísticamente queden aposentados en la cúspide de la raíz para minimizar latencias.

- **Paso 1 (Subproblema):** Sea $C[i][j]$ la cota esperable **mínima** de coste de sondeo derivado de consolidar un ABBO ensamblado lícitamente con un segmento de claves desde $k_i \dots k_j$. Complementariamente, sea $P[i][j]$ la acumulación aritmética de las probabilidades frecuenciales endémicas de dichas claves contenidas.
- **Paso 2 (Recurrencia):** Enaltecemos arbitrariamente una clave nominal r (alojada en la cota $i \leq r \leq j$) a la corona de raíz.estructural del sub-injerto. Inexorablemente, este alzamiento fractura el dominio, induciendo un coste de las frondas inferiores. Y aquí reside el toque maestro: al engarzar estos subárboles bajo el amparo de la nueva raíz r , la totalidad orgánica de sus nodos se hunde catastróficamente un nivel de profundidad $\rightarrow$ Esta penalización colateral ineludible suma +1 al coste general ponderado de cada nodo, lo que matemáticamente equivale a inyectar la carga de probabilidad total $P[i][j]$ a la factura final.

$$C[i][j] = \min_{i \leq r \leq j} \{C[i][r-1] + C[r+1][j] + P[i][j]\}$$

- **Paso 3 (Casos Base):** La purga de la trivialidad: $C[i][i-1] = 0$ (un ramaje vacío extingue el coste) y $C[i][i] = prob_i$ (un árbol raquíutico de un solo nodo asume netamente su probabilidad innata).
- **Paso 4 (Orden de Evaluación):** Clónica del modus operandi matricial del problema C. Crecimiento por diagonales secuenciales incrementando el abanico diametral de L .
- **Paso 5 (Complejidad y Reconstrucción):** Coste de $\mathcal{O}(N^3)$. Se guarda el índice majestuoso $Raiz[i][j] = r$. Para la orquestación reconstructiva final, la llamada recurrente establece a r como patriarca; asignando al linaje izquierdo el nodo extraído de la consulta a $[i][r-1]$, y cediendo al linaje derecho la custodia del trono extraído de $[r+1][j]$.

7. Catálogo Analítico de los Ejemplos de las Diapositivas

Para asegurar que reconoces cualquier variante vista en las clases de teoría, aquí tienes tabulados los problemas avanzados del material de diapositivas con su respectivo patrón de resolución.

■ 1. Grafo Multietapa (Tema 3 del Índice)

- *Patrón:* A / E (Unidimensional saltando por etapas o columnas" del grafo).
- *Desarrollo:* El objetivo es hallar el camino mínimo desde un nodo origen (en la etapa 1) hasta un sumidero final. Se calcula construyendo la matriz hacia atrás. La recurrencia establece que el coste desde un nodo i es igual al coste de la arista hacia un nodo adyacente l de la siguiente etapa, más el coste previamente calculado desde l hasta el final: $coste[i][j] = \min(c(j, l) + coste[l+1][j])$.

■ 2. Fiabilidad de Sistemas (Tema 8 del Índice)

- *Patrón:* B (Variante de Mochila multiobjetivo limitando presupuesto).
- *Desarrollo:* Se desea maximizar la fiabilidad global (el producto de las probabilidades de no fallo) de un sistema compuesto por N dispositivos en serie. Para evitar fallos, se compran m_i copias redundantes de cada dispositivo sin exceder un presupuesto máximo C .
La recurrencia prueba invertir presupuesto en m_i copias del dispositivo actual:
$$F[i][c] = \max_{m_i} \{ \phi_i(m_i) \cdot F[i-1][c - m_i \cdot \text{coste}_i] \}.$$

■ 3. Problema del Viajante de Comercio - TSP (Tema 9 del Índice)

- *Patrón:* Máscara de Bits (Bitmask DP). Estado = Nodo actual + Subconjunto de visitados.
- *Desarrollo:* Resolver el ciclo hamiltoniano mínimo. Fuerza bruta exige evaluar $\mathcal{O}(N!)$ permutaciones. Con PD, memorizamos estados $g(i, S)$ que representan el coste mínimo desde el nodo i , pasando obligatoriamente por todos los nodos del conjunto S , y terminando en el nodo origen 1.
$$g(i, S) = \min_{j \in S} \{ C_{ij} + g(j, S - \{j\}) \}.$$
 Reduce drásticamente el coste a $\mathcal{O}(N^2 2^N)$.

■ 4. Triangulación de Polígonos (Tema 10 del Índice)

- *Patrón:* D (Idéntico a Multiplicación de Matrices - Intervalos desvinculados).
- *Desarrollo:* Encontrar cómo dividir un polígono convexo en triángulos minimizando el coste (ej. la suma de perímetros). La tabla $T[i][j]$ almacena el coste de triangular el subpolígono formado por los vértices desde i hasta j . Se corta iterando un vértice pivote k para formar un triángulo base (i, k, j) .
$$T[i][j] = \min_{i < k < j} \{ T[i][k] + T[k][j] + \text{coste_triangulo}(i, k, j) \}.$$
 Se evalúa por diagonales.

■ 5. Planificación de Trabajos (Tema 11 del Índice)

- *Patrón:* Variante Unidimensional con Pre-Ordenamiento temporal.
- *Desarrollo:* Dados N trabajos con una duración, un deadline y un beneficio, se busca maximizar la ganancia sin solapar trabajos. Se requiere ordenar previamente los trabajos por fecha de finalización y aplicar una decisión binaria iterativa ("Lo ejecuto y sumo su valor, o lo descarto").

■ 6. Una Competición Internacional (Tema 12 del Índice)

- *Patrón:* C (Matriz Bidimensional Probabilística sin recursión de coste).
- *Desarrollo:* Se plantea tabular probabilidades matemáticas puras (no costes). $P(i, j)$ es la probabilidad de que el Equipo A gane el torneo global si a él le faltan i victorias para lograrlo, y al Equipo B le faltan j victorias. La fórmula dictamina: $P(i, j) = p \cdot P(i-1, j) + q \cdot P(i, j-1)$ (donde p es la prob. de ganar un único partido y q la prob. de perderlo). La matriz se completa de derecha a izquierda y de abajo arriba.